To design a robust, secure, and scalable solution for this scenario, we must implement a **Three-Tier Architecture**. This design separates the application into three functional layers: the Presentation (Public) tier, the Application (Private) tier, and the Data (Private) tier.

The Architecture Diagram Components

Your diagram should include the following AWS service icons, organized within a **VPC** spanning at least two **Availability Zones (AZs)** for High Availability:

1. **Networking Layer:**
    - **Amazon VPC:** The isolated virtual network.
    - **Internet Gateway:** To allow communication between the VPC and the internet.
    - **Public Subnets:** To house the Application Load Balancer.
    - **Private Subnets:** To house the EC2 Web Servers and the RDS Database instances.
2. **Compute Tier:**
    - **Amazon EC2:** Web servers running the application logic.
    - **Auto Scaling Group (ASG):** To automatically add or remove EC2 instances based on traffic.
3. **Load Balancing:**
    - **Application Load Balancer (ALB):** To distribute incoming web traffic across the EC2 instances.
4. **Database Tier:**
    - **Amazon RDS (MySQL):** A managed database service.
    - **Multi-AZ Deployment:** A primary database in one AZ and a standby in another for failover protection.

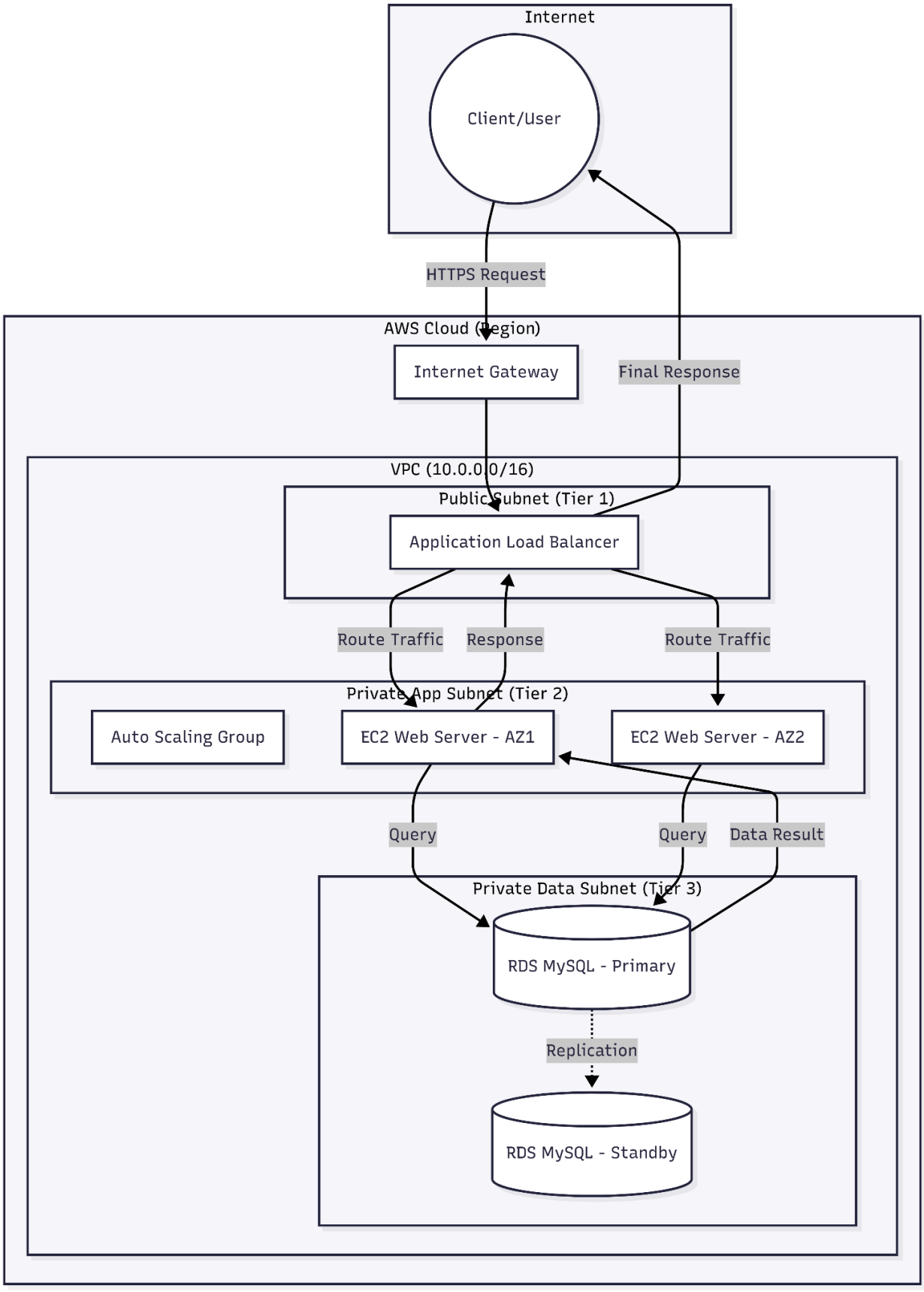
Architecture Justification

I chose a **Three-Tier Architecture** because it follows the AWS Well-Architected Framework's best practices for security and reliability. By placing the EC2 instances and the RDS database in **Private Subnets**, we ensure they are not directly accessible from the internet, significantly reducing the attack surface. Only the **Application Load Balancer** sits in the Public Subnet to receive external requests.

For the compute layer, I used **Amazon EC2** within an **Auto Scaling Group**. This ensures that if a server fails or traffic spikes, the system automatically launches new instances to maintain performance. For the data layer, **Amazon RDS with Multi-AZ** was selected to provide high availability. If the primary MySQL database fails, RDS automatically fails over to the standby instance in a different Availability Zone, ensuring the web application remains functional without manual intervention.

The Flow of Data

1. **Request:** The client sends an HTTPS request over the internet. It enters the VPC through the **Internet Gateway**.
2. **Load Balancing:** The request hits the **Application Load Balancer (ALB)**. The ALB inspects the traffic and routes it to one of the healthy **EC2 instances** in the private subnets.
3. **Processing:** The EC2 instance processes the request. To fulfill the query, it sends a database request to the **Amazon RDS MySQL** endpoint.
4. **Database Query:** The RDS instance retrieves the requested data and sends it back to the EC2 instance.
5. **Response:** The EC2 instance formats the data into a response and sends it back through the ALB.
6. **Delivery:** The ALB sends the final response back to the client via the Internet Gateway.


Internet
Client/User
HTTPS Request
AWS Cloud (Region)
Internet Gateway
Final Response
VPC (10.0.0.0/16)
Public Subnet (Tier 1)
Application Load Balancer
Route Traffic
Response
Route Traffic
Private App Subnet (Tier 2)
Auto Scaling Group
EC2 Web Server - AZ1
EC2 Web Server - AZ2
Query
Query
Data Result
Private Data Subnet (Tier 3)
RDS MySQL - Primary
Replication
RDS MySQL - Standby